

Laboratory report of Nanjing University of Information Science and Technology

Experiment Title Distributed Object Storage & Fault Tolerance Testing

Data Applied Computing Class Cooperative 1 Name Lu Moyou ID 202483910013

一、Experimental purpose

Master Object Storage: Understand the difference between traditional file paths and S3-compatible Bucket/Object structures.

Experience Fault Tolerance: Observe how data remains accessible even when physical "disks" are destroyed.

二、Experiment Contents

In this experiment, students will simulate a distributed cloud object storage system similar to: Amazon S3, Alibaba Cloud OSS

- using: MinIO, Docker Compose
- Students will:
 - Deploy a 4-disk distributed MinIO cluster
 - Upload and manage cloud objects
 - Observe backend storage fragmentation
 - Simulate storage node failure
 - Verify distributed storage fault tolerance

三、Detailed steps of the experiment

Lab Report Requirements: Clearly present the procedure step by step, with corresponding screenshots provided for each stage. **Screenshots must demonstrate that they were taken on your own computer**, for example by including visible system time, desktop environment, or other identifiable personal elements.

Step 1: Deploy a 4 - Node Distributed Cluster

In your Codespaces terminal, create a dedicated directory for this lab:

```
mkdir minio - lab && cd minio - lab
```

A screenshot of a terminal window with a light blue header bar containing tabs for '问题', '输出', '调试控制台', '终端', and '端口'. The '终端' tab is active. The terminal shows the command '@lumoyou → /workspaces/cloud-computing-lab-github (main) \$ mkdir minio - lab && cd minio - lab' being executed.

Step 2: Create Storage Directories

Create four independent storage directories to simulate four storage disks.

```
mkdir -p data1 data2 data3 data4
```

Verify:

Ls

A screenshot of a terminal window showing the command '@lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) \$ ls' being executed. The output 'data1 data2 data3 data4' is displayed in green text.

Step 3: Create docker-compose.yml

Create a file named:

docker-compose.yml in minio-lab

with the following content:

```
services:
```

minio:

image: quay.io/minio/minio

container_name: cloud-minio

volumes:

- ./data1:/data1
- ./data2:/data2
- ./data3:/data3
- ./data4:/data4

ports:

- "9000:9000"
- "9001:9001"

environment:

MINIO_ROOT_USER: admin

MINIO_ROOT_PASSWORD: password123

command: server /data{1...4} --console-address
":9001"

mc:

image: minio/mc

container_name: cloud-mc

```

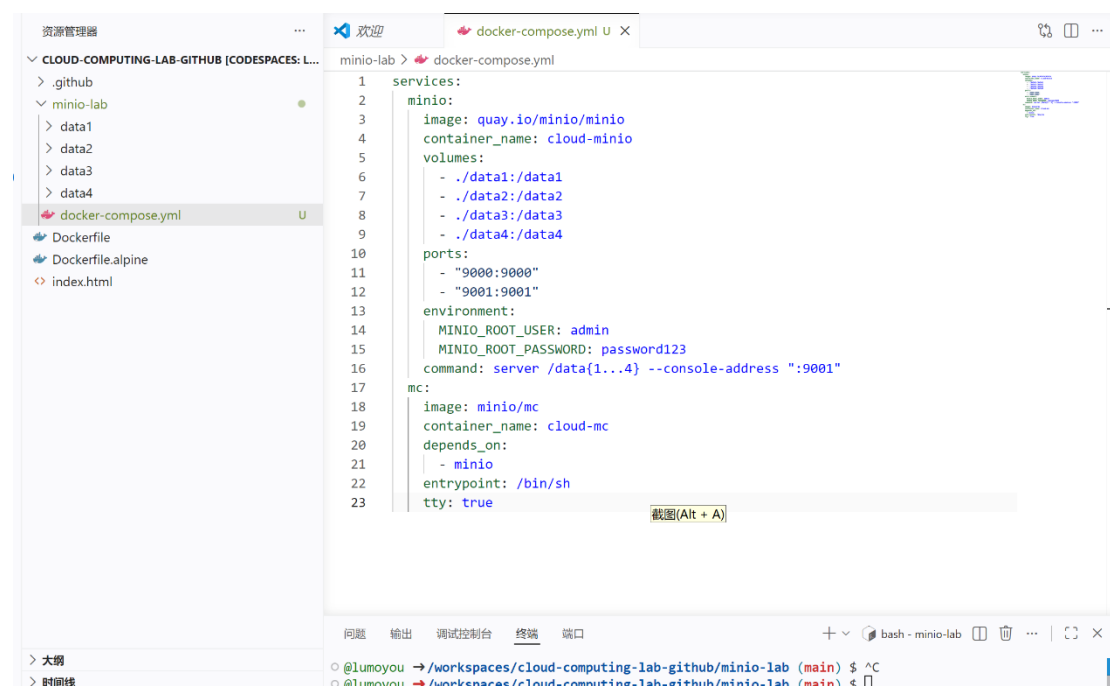
depends_on:

  - minio

entrypoint: /bin/sh

tty: true

```



Step 4: Explain the docker-compose.yml File

Students should explain the meaning of the following components:

Component	Meaning
image	Docker image used to create the container
container_name	The name of the container, for easy management
volumes	Mount the host machine's directory into the container to achieve data persistence
ports	Map the container port to the host machine for service access
environment	Set environment variables inside the container, such as username and password
command	Start MinIO distributed storage service
depends_on	Specify the container startup order, with mc depending on minio starting first

Step 5: Launch the Distributed Cluster

Start all services:

cd minio-lab

docker compose up -d

Verify running containers:

docker ps

You should see:

cloud-minio

cloud-mc

```
@lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker compose up -d
[+] Running 16/16
  ✓ mc Pulled                                6.4s
  ✓ minio Pulled                             7.2s

[+] Running 3/3
  ✓ Network minio-lab_default Created          0.1s
  ✓ Container cloud-minio Started             4.9s
  ✓ Container cloud-mc Started                1.3s

@lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ ^C
@lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
9a305b5d8125	minio/mc	"/bin/sh"	10 seconds ago	Up 8 seconds	
e4dcc9e6ce1f	quay.io/minio/minio	"/usr/bin/docker-ent..."	14 seconds ago	Up 9 seconds	0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp

Step 6: Configure MinIO Client Alias

Configure the MinIO client (mc) to connect to the MinIO server.

docker exec cloud-mc mc alias set myminio http://minio:9000 admin

password123

Observation:

You should see: Added myminio successfully.

```
@lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123
Added `myminio` successfully.
@lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $
```

Step 7: Create a Bucket

Create a cloud storage bucket.

```
docker exec cloud-mc mc mb myminio/my-cloud-storage
```

```
@lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc mb myminio/my-cloud-storage
Bucket created successfully `myminio/my-cloud-storage`.
```

Step 8: Create and Upload a Large Test File

In distributed object storage systems, large files are automatically divided into multiple fragments and distributed across different storage disks.

This step demonstrates the Erasure Coding and distributed storage mechanism.

8.1 Create a Large Test File

Generate a 20 MB random binary file:

```
dd if=/dev/urandom of=bigfile.bin bs=1M count=20
```

8.2 Verify the File

Check the file size:

```
ls -lh bigfile.bin
```

8.3 Copy the File into the mc Container

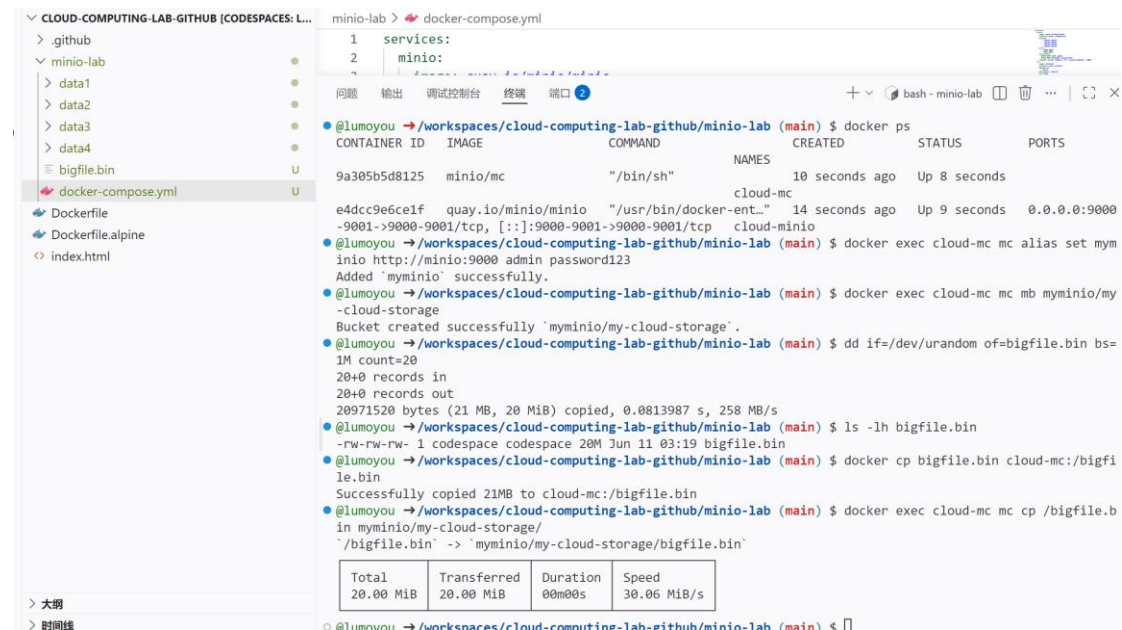
```
docker cp bigfile.bin cloud-mc:/bigfile.bin
```

8.4 Upload the File to Cloud Storage

```
docker exec cloud-mc mc cp /bigfile.bin myminio/my-cloud-storage/
```

Observation:

You should see upload progress information.



The screenshot shows a terminal window with the following commands and output:

```
minio-lab > docker-compose.yml
1 services:
2   minio:
3     image: minio/minio
4     ports:
5       - 9000:9000
6       - 9001:9001
7     command:
8       - server /data
9       - /usr/bin/docker-entrypoint.sh
10    environment:
11      - MINIO_ROOT_USER=admin
12      - MINIO_ROOT_PASSWORD=password123
13    volumes:
14      - data1:/data
15      - data2:/data
16      - data3:/data
17      - data4:/data
```

```
@lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
9a305b5d8125   minio/mc      "/bin/sh"               10 seconds ago Up 8 seconds  cloud-mc
e4dcc9e6ce1f   quay.io/minio/minio "/usr/bin/docker-ent..." 14 seconds ago Up 9 seconds  0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp cloud-minio
```

```
@lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc alias set myminio http://minio:9000 admin password123
Added 'myminio' successfully.
```

```
@lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc mb myminio/my-cloud-storage
Bucket created successfully 'myminio/my-cloud-storage'.
```

```
@lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ dd if=/dev/urandom of=bigfile.bin bs=1M count=20
20+0 records in
20+0 records out
20971520 bytes (21 MB, 20 MiB) copied, 0.0813987 s, 258 MB/s
```

```
@lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ ls -lh bigfile.bin
-rw-rw-rw- 1 codespace codespace 20M Jun 11 03:19 bigfile.bin
```

```
@lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker cp bigfile.bin cloud-mc:/bigfile.bin
Successfully copied 21MB to cloud-mc:/bigfile.bin
```

```
@lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc cp /bigfile.bin myminio/my-cloud-storage/
'/bigfile.bin' -> 'myminio/my-cloud-storage/bigfile.bin'
```

Total	Transferred	Duration	Speed
20.00 MiB	20.00 MiB	00m00s	30.06 MiB/s

```
@lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $
```

Step 9: Verify Object Upload

List all objects in the bucket.

```
docker exec cloud-mc mc ls myminio/my-cloud-storage/
```

You should see: bigfile.bin



```
@lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker exec cloud-mc mc ls myminio/my-cloud-storage/
[2026-06-11 03:19:42 UTC] 20MiB STANDARD bigfile.bin
```

Step 10: Observe Backend Storage Fragmentation

Observe how MinIO distributes data across multiple storage disks.

Run:

```
ls -R ./data1
```

Then:

```
ls -R ./data2
```

Then:

```
ls -R ./data3
```

Then:

```
ls -R ./data4
```

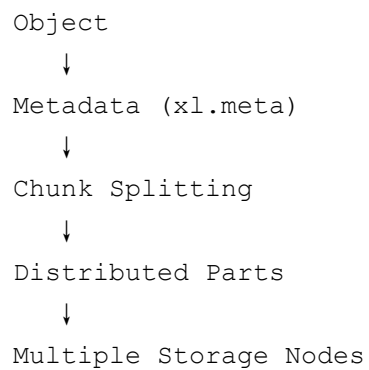
Observation:

Students should notice:

- object metadata
- fragmented storage structure
- distributed object placement

Students must compare the structure across all disks

Cloud Object Storage Architecture




```

• @lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ ls -R ./data1
./data1:
my-cloud-storage

./data1/my-cloud-storage:
bigfile.bin

./data1/my-cloud-storage/bigfile.bin:
ed733d1a-5b0f-47ef-8d22-45c221d0a0ab  xl.meta

./data1/my-cloud-storage/bigfile.bin/ed733d1a-5b0f-47ef-8d22-45c221d0a0ab:
part.1  part.2
• @lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ ls -R ./data2
./data2:
my-cloud-storage

./data2/my-cloud-storage:
bigfile.bin

./data2/my-cloud-storage/bigfile.bin:
ed733d1a-5b0f-47ef-8d22-45c221d0a0ab  xl.meta

./data2/my-cloud-storage/bigfile.bin/ed733d1a-5b0f-47ef-8d22-45c221d0a0ab:
part.1  part.2

• @lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ ls -R ./data3
./data3:
my-cloud-storage

./data3/my-cloud-storage:
bigfile.bin

./data3/my-cloud-storage/bigfile.bin:
ed733d1a-5b0f-47ef-8d22-45c221d0a0ab  xl.meta

./data3/my-cloud-storage/bigfile.bin/ed733d1a-5b0f-47ef-8d22-45c221d0a0ab:
part.1  part.2
• @lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ ls -R ./data4
./data4:
my-cloud-storage

./data4/my-cloud-storage:
bigfile.bin

./data4/my-cloud-storage/bigfile.bin:
ed733d1a-5b0f-47ef-8d22-45c221d0a0ab  xl.meta

./data4/my-cloud-storage/bigfile.bin/ed733d1a-5b0f-47ef-8d22-45c221d0a0ab:
part.1  part.2

```

Step 11: Simulate Disk Failure

Distributed cloud storage systems must tolerate hardware failures.

11.1 Stop the Cluster

docker compose down

11.2 Delete One Storage Disk

sudo rm -rf data3

mkdir data3

This simulates disk failure.

11.3 Restart the Cluster

docker compose up -d

```
@lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker compose down
[+] Running 3/3
  ✓ Container cloud-mc      Removed      10.3s
  ✓ Container cloud-minio   Removed      0.3s
  ✓ Network minio-lab_default Removed      0.0s
@lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ rm -rf data3
@lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ mkdir data3
@lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ docker compose up -d
[+] Running 3/3
  ✓ Network minio-lab_default Created      0.1s
  ✓ Container cloud-minio   Started     0.4s
  ✓ Container cloud-mc      Started     0.7s
@lumoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ █
```

Step 12: Verify Fault Tolerance

Check whether objects are still accessible.

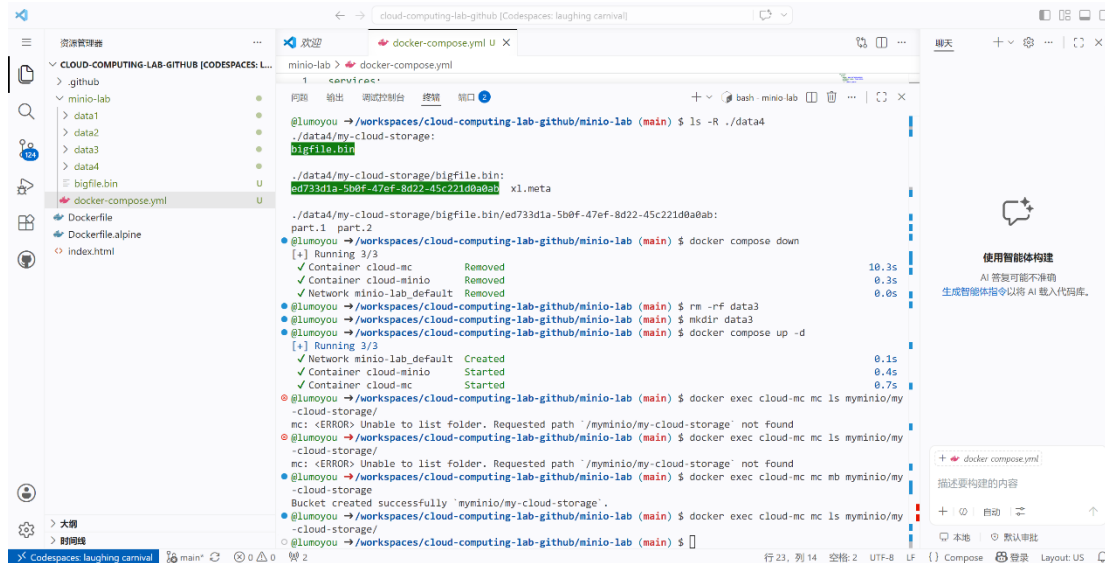
docker exec cloud-mc mc ls myminio/my-cloud-storage/

Observation:

bigfile.bin should still exist.

This demonstrates:

Erasur Coding and distributed redundancy



Step 13: Use git to save your changes.

```

@lunoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ git init
Initialized empty Git repository in /workspaces/cloud-computing-lab-github/minio-lab/.git/
@lunoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ git add .
@lunoyou → /workspaces/cloud-computing-lab-github/minio-lab (main) $ git commit -m "Complete MinIO fault tolerance lab"
[main (root-commit) 134899b] Complete MinIO fault tolerance lab
148 files changed, 27 insertions(+)
create mode 100644 bigfile.bin
create mode 100644 data1/.minio.sys/buckets/.background-heal.json/xl.meta
create mode 100644 data1/.minio.sys/buckets/.bloomcycle.bin/xl.meta
create mode 100644 data1/.minio.sys/buckets/.usage-cache.bin.bkp/xl.meta
create mode 100644 data1/.minio.sys/buckets/.usage-cache.bin/xl.meta
create mode 100644 data1/.minio.sys/buckets/.usage.json/xl.meta
create mode 100644 data1/.minio.sys/buckets/my-cloud-storage/.metadata.bin/xl.meta
create mode 100644 data1/.minio.sys/buckets/my-cloud-storage/.usage-cache.bin.bkp/xl.meta
create mode 100644 data1/.minio.sys/buckets/my-cloud-storage/.usage-cache.bin/xl.meta
create mode 100644 data1/.minio.sys/config/config.json/xl.meta
create mode 100644 data1/.minio.sys/config/iam/format.json/xl.meta
create mode 100644 data1/.minio.sys/format.json
create mode 100644 data1/.minio.sys/pool.bin/xl.meta
create mode 100644 data1/.minio.sys/tmp/.trash/.writable-check-26cfcdded-6e6b-49d3-8a23-b5696c2c7e59.tmp
create mode 100644 data1/.minio.sys/tmp/.trash/.writable-check-bfb0a977-c55e-418a-8ac7-eb68711f0ddf.tmp
create mode 100644 data1/.minio.sys/tmp/.trash/0205fbad-a624-42ee-93cc-56211ed4e9a5/xl.meta.bkp
create mode 100644 data1/.minio.sys/tmp/.trash/0f5e347c-f4cb-4b30-a8ae-4a49d93c260b/xl.meta.bkp
create mode 100644 data1/.minio.sys/tmp/.trash/143b24e4-afe8-4542-b397-3c51763d4582/xl.meta.bkp
create mode 100644 data1/.minio.sys/tmp/.trash/297a4e01-a4a6-4e08-9eb6-03fe4a4596e6/xl.meta.bkp
create mode 100644 data1/.minio.sys/tmp/.trash/4053383e-163f-4ea2-bc0c-65c03b5f67d8/xl.meta.bkp
create mode 100644 data1/.minio.sys/tmp/.trash/611ea69e-6cc1-460c-a9d3-9df6ecda93f0/xl.meta.bkp
create mode 100644 data1/.minio.sys/tmp/.trash/764cfea5-24c7-46c5-9db1-b27e18d5bb4d/xl.meta.bkp
create mode 100644 data1/.minio.sys/tmp/.trash/825dd276-5abe-4a08-8aad-eef6a7667cd3/xl.meta.bkp
create mode 100644 data1/.minio.sys/tmp/.trash/85934539-35be-4292-82a7-c879505a4811/xl.meta.bkp
create mode 100644 data1/.minio.sys/tmp/.trash/8ef2d5e8-57d6-4fdc-b18c-8def7ec60d68/xl.meta.bkp

```

四、Reflection

1. Problems Encountered and Proposed Solutions

During the experiment, a common issue is that the mc container fails to

connect to the MinIO server due to startup order. This can be resolved by adding `depends_on` and using a healthcheck or simply restarting the mc container after the MinIO service is fully ready. Another issue is permission errors when deleting data3 using `sudo rm -rf`, which can be avoided by ensuring the directory is owned by the current user or using `sudo` correctly.

2. Explain in 3–5 sentences why data is still available after deleting one storage directory.

MinIO uses erasure coding to split objects into data and parity fragments distributed across multiple disks. Even if one disk (e.g., data3) is lost, the remaining fragments contain enough information to reconstruct the original object. This redundancy ensures data availability and durability without requiring a complete copy of the object. As long as the number of lost disks is within the erasure coding tolerance limit, the system can still serve the data successfully.